E-MUNSHI

Product Architecture, ADRs, Budget & 30-Day Pilot Roadmap

Production-oriented SaaS plan for an India-first, Tally-integrated AI finance product

PROJECT PARAMETERS
Reference competitor: Riko AI (https://rikoai.in/) | Current prototype: E-Munshi (http://emunshie.com/) | Reference repository: https://github.com/dhananjay1405/tally-mcp-server.git | Build budget: ■50,000–■1,00,000 | Deadline: 30 days | Goal: working pilot with real users and financial files | Initial channel: Web application | Team: existing engineering team (skills/capacity to confirm)

Prepared: 23 October 2026

Purpose: consolidate product direction, proposed architecture, technical decisions, costs, delivery plan, security requirements, and immediate next steps.

1. Executive Summary

E-Munshi should be built as a production-oriented, multi-tenant SaaS web application with a Windows-based Tally connector, a deterministic financial data engine, and AI-assisted workflows. The first-month goal should be a controlled pilot—not full parity with every Riko AI feature.

Recommended foundation: TypeScript modular monolith, PostgreSQL, narrowly scoped Tally adapter based on the supplied MCP repository, deterministic calculations for authoritative financial values, and AI orchestration limited to approved tools.

CORE PRINCIPLE
AI interprets questions and explains verified results. Deterministic application code calculates authoritative figures. Every answer should be traceable to a company, reporting period, data source, and calculation.

1.1 Project inputs

Item	Current understanding
Reference competitor	Riko AI — https://rikoai.in/
Existing prototype	E-Munshi — http://emunshie.com/
Tally integration candidate	https://github.com/dhananjay1405/tally-mcp-server.git
Scope	Tally integration, finance chat, dashboards and reports
One-time budget	■50,000–■1,00,000
Team	Existing team; skills/capacity to confirm
Prototype stack	Unknown; audit required
Deadline / goal	30 days; working pilot
Initial channel	Web application

1.2 Success definition

- A small pilot group can securely sign in, connect a supported TallyPrime installation, select a company, and retrieve agreed reports.
- Financial figures reconcile with reference outputs from Tally for agreed test periods.
- Supported natural-language questions return tool-backed answers with source information and freshness indicators.
- Cross-tenant access is blocked and tested; synchronization failures are recoverable.
- The team can deploy, monitor, support, and update the product without relying on an opaque AI-generated codebase.

1.3 Important constraints

- Budget is plausible only for constrained scope and an existing team/founder-led engineering; not a guarantee of market-rate agency delivery.
- One month is aggressive for a robust live Tally connector, SaaS backend, AI workflows, and security testing. Parallel work and tight scope are essential.
- Review the repository's source, license, dependencies, and runtime behaviour before deciding how to reuse it.

2. Product and Competitor Understanding

Public positioning indicates Riko AI is more than a chatbot: it focuses on an AI-powered finance/accounting layer connected to Tally, with financial questions, reports, workpacks, approvals, and generated deliverables. E-Munshi's current prototype appears more upload-oriented, emphasizing Excel/CSV and financial records, validation, and insights.

Product area	Potential E-Munshi requirement
Tally integration	Read/synchronize accounting data; identify installation/company; monitor connection health.
AI finance chat	Plain-English questions answered from real financial records.
Reports and dashboards	Sales, P&L, balance sheet, cash position, receivables, payables, inventory and ledger details.

Product area	Potential E-Munshi requirement
Accounting workpacks	MIS generation, bank/GST reconciliation and month-end workflows, phased by priority.
Approvals and actions	Review, edit, approve/reject proposed actions; no write-back in initial pilot.
Document generation	Excel/CSV/PDF deliverables where practical.
Notifications	Payment reminders and potentially WhatsApp workflows later.
Security and auditability	Company isolation, permissions, source tracing, audit events and controlled access.

Reference links: https://rikoai.in/ ; https://rikoai.in/features ; https://rikoai.in/faq. Public pages are useful for discovery but should be rechecked directly during audit; exact feature parity should not be assumed from marketing pages alone.

2.1 The key product distinction

E-Munshi's prototype appears oriented toward file uploads, validation, analytics, and AI finance chat. Riko's broader positioning emphasizes ongoing Tally connectivity, accounting queries, workpacks, approvals, and deliverables. This distinction affects architecture, time, and cost.

PRODUCT STRATEGY
Build the smallest reliable end-to-end Tally-connected workflow first. Add feature parity incrementally after report accuracy, connector reliability, tenant security, and the core user journey are proven.

2.2 Suggested pilot scope

P0 — Must ship

- Organization/user onboarding; sign-in; roles; company selection.
- Secure Windows connector pairing and connection-health status.
- Tally retrieval for a defined set of reports.
- Trial balance, P&L, balance sheet, receivables/payables, and ledger statements, subject to connector validation.
- Financial dashboard with date filters, source information, loading/error/empty states.
- Natural-language questions mapped to approved report tools.
- CSV/Excel exports where practical.
- Audit logs, retries, backups, monitoring, and automated tests.

P1 — Only after P0 is reliable

- One polished workpack, such as monthly MIS.
- Cached reporting data for faster repeat queries.
- Chat follow-ups and saved reports.
- Ledger drill-down and overdue summaries.
- Connection diagnostics for users and support.

P2 — Post-pilot roadmap

- Tally write-back and voucher creation.
- Automated bank/GST reconciliation with exception handling.
- Full workpack catalogue.
- WhatsApp, mobile apps, subscription/billing automation.
- Enterprise SSO, advanced CA-firm administration, and analytics.

3. Using the Tally MCP Server Repository

Repository: https://github.com/dhananjay1405/tally-mcp-server.git

The published README describes a Node.js/TypeScript MCP server that communicates with TallyPrime through XML-over-HTTP. Publicly described tools include trial balance, P&L, balance sheet, outstanding bills, ledger statements, stock balances, and master data. Verify exact implementation against current source.

3.1 What MCP means

MCP exposes tools to compatible AI clients. It is not a complete multi-tenant SaaS backend or secure customer accounting platform. The web product can use typed APIs and an internal adapter for reports/calculations; the whole product need not make an MCP round-trip for every calculation.

3.2 Recommended reuse approach

1. Clone and inspect the repository, run it locally, map tools to source, and identify the Tally communication layer.

2. Review license and dependency terms before adapting/distributing code.

3. Review remote authentication, secrets, company selection, caching, error handling and write-capable tools.

4. Prefer extracting reusable Tally communication/report logic into a dedicated connector package with a narrow read-only interface.

5. Fallback: run MCP as an internal service and expose only explicitly permitted tools through the authenticated backend.

6. Build E-Munshi's own tenant management, roles, permissions, chat history, calculations, jobs, audit logs and public APIs.

The README describes deployment options including an SSH tunnel. Do not expose Tally's local port 9000 publicly or treat an MCP password page as SaaS authentication.

3.3 What must be validated

- Repository license and permission to adapt/distribute.
- Node.js/TypeScript versions, dependency health, build/test status and maintenance activity.
- Tally XML request/response behaviour and compatibility with target TallyPrime configurations.
- Company selection and tenant-isolation behaviour.
- Whether read tools are read-only and how write tools are gated.
- Authentication, credential handling, logging, retry logic, timeouts and caching.
- Data formats, numeric precision, date semantics, debit/credit conventions and report reconciliation.

SECURITY DECISION
Disable all write-capable Tally tools in the pilot. Connector initiates authenticated outbound connection to the cloud backend; never expose the customer's Tally XML port directly to the public internet.

4. Target Technical Architecture

4.1 Logical architecture

Layer	Recommended component	Responsibilities
Presentation	Next.js + TypeScript	Dashboard, chat, reports, company switcher, onboarding, approvals and exports.
Application API	NestJS + TypeScript	Authentication/authorization, tenant context, validation, rate limits, business APIs and orchestration.
Financial engine	TypeScript module	Validated calculations, report definitions, reconciliations, source references and deterministic totals.
AI orchestration	Provider-independent service	Intent interpretation, approved tool selection, structured outputs, explanations, timeouts and budgets.
Application database	PostgreSQL via Supabase	Organizations, users, memberships, connections, report metadata, jobs, chat history and audit events.
Background jobs	Persistent worker + PostgreSQL-backed queue (e.g. pg-boss)	Synchronization, report generation, retries, idempotency and failure states.
File storage	Private object storage	Generated reports and exports; short-lived download links.
Local integration	Packaged Windows Node.js/TypeScript connector	Talk to local Tally XML interface; secure pairing, heartbeats, retries and read-only retrieval.
Deployment	Vercel/suitable web host; managed Node.js API/worker host	HTTPS, environment separation, logs, deploys, monitoring and recovery.

4.2 Recommended request/data flow

1. User signs in and selects an authorized organization and accounting company.

2. Frontend sends a request to authenticated backend; backend derives organization permissions from session.

3. Backend validates inputs and selects an allowlisted report tool.

4. Connector retrieves data from Tally, or a sufficiently fresh validated cache is used.

5. Financial engine calculates totals and records source metadata, period and freshness.

6. AI explains structured results without being the authority for numeric calculations.

7. UI displays answer, company, as-of date, source context and freshness warnings.

8. Audit events and job state are recorded.

4.3 Deployment guidance

- Frontend: Vercel or suitable application host.
- API: managed Node.js host; share with frontend only if runtime/resource limits permit.
- Database/authentication: Supabase/PostgreSQL.
- Worker: persistent process; do not place long-running Tally connections in ephemeral serverless functions.
- Connector: packaged, versioned Windows application on customer computer.
- Separate development and production environments, credentials, migrations and releases.

4.4 Suggested project structure

A monorepo is practical for a small TypeScript team; shared types and validation schemas can be reused without introducing microservice operations.

- apps/web — Next.js UI
- apps/api — NestJS API
- apps/worker — durable background jobs
- apps/tally-connector — Windows connector
- packages/shared — shared schemas, types, error codes and validation
- packages/tally-adapter — Tally communication and report adapters
- packages/finance-engine — deterministic calculations and report definitions
- packages/ai-orchestrator — model interface, tool registry and structured outputs
- infra/ — deployment configuration, environment templates and runbooks
- docs/adr/ — accepted architecture decision records
- tests/fixtures/ — sanitized reference datasets and Tally report fixtures

5. Architecture Decision Records (ADRs)

Proposed decisions for review and acceptance during the first two days. Record owner, date, status and any superseding ADR.

ADR-001 — Modular monolith, not microservices

Decision: one backend deployable with clear modules; separate connector and worker processes where runtime needs them. Rationale: small team, one-month timeline, fewer deployment and debugging costs. Revisit only when measured scale, isolation, or team boundaries require it.

ADR-002 — Reuse Tally integration logic, not the whole product

Decision: adapt reusable Tally communication and report tools behind a narrow internal interface. Rationale: avoid rebuilding integration logic while owning product security, tenancy, workflows, and validation. License/dependency review is a prerequisite.

ADR-003 — PostgreSQL for application system of record

Decision: use relational tables for tenant membership, connection metadata, chat, report metadata, job state and audit events. Rationale: integrity and transactional updates. Do not use a vector database as the primary accounting database.

ADR-004 — Deterministic financial engine

Decision: AI selects approved tools and explains results; application code computes authoritative totals. Rationale: model output can be plausible but wrong. Preserve company, reporting period, calculation definition, source and freshness for metrics.

ADR-005 — Outbound connector; read-only pilot

Decision: connector makes authenticated outbound requests; Tally stays on the customer's Windows machine; write operations disabled. Rationale: avoid public exposure of Tally and limit risk while data correctness and approvals are tested.

ADR-006 — Organization and company are distinct entities

Decision: model organizations, memberships, Tally installations and accounting companies separately. Rationale: CA firms and groups may manage multiple clients/companies. Company name is not a security identifier.

ADR-007 — PostgreSQL-backed jobs before Redis

Decision: start with durable jobs in PostgreSQL, retries, idempotency and visible failure states. Rationale: report jobs must survive restarts, while a separate Redis queue adds operations. Revisit after measuring throughput.

ADR-008 — Provider-independent AI service

Decision: define an internal interface for model calls, structured output, tool permissions, token budgets and timeouts. Rationale: models/pricing change; provider can be changed without rewriting product logic. Minimize raw ledger data in prompts.

ADR-009 — Approval and audit are first-class

Decision: proposed actions record creator, scope, payload, approval state and execution history; no Tally writes in initial pilot. Rationale: UI approval buttons are not enough; server must verify permissions and prevent replay.

ADR-010 — Reuse a supported UI component library

Decision: use accessible components and consistent design tokens, adapting the prototype where appropriate. Rationale: prioritize accounting correctness and integration over rebuilding generic UI primitives.

6. Financial Data, AI and Tool Design

6.1 Initial allowlisted tools

Tool	Required inputs / behaviour
get_profit_loss	Authorized company, from_date, to_date; structured totals and source metadata.
get_trial_balance	Authorized company and period/as-of parameters; agreed accounting conventions.
get_outstanding_bills	Authorized company, receivable/payable nature, as-of date; validated ageing where available.
get_ledger_statement	Authorized company, authorized ledger ID, from_date, to_date.
get_stock_balance	Authorized company, optional item ID, as-of date; depends on Tally data/configuration.

The server derives organization and permissions from the authenticated session. It must not trust a tenant ID supplied by model/browser as proof of authorization. Avoid arbitrary model-generated SQL in version one.

6.2 Example workflow: “Who owes us the most money?”

1. Authenticate; verify organization, role, company and reporting date.

2. Retrieve outstanding bills from Tally or validated cache within freshness policy.

3. Calculate and sort balances in deterministic code.

4. Pass structured results to AI for a concise explanation.

5. Display answer, as-of date, source information and freshness warning if needed.

6.3 Data storage strategy

- Store application data, connection records, report metadata and audit logs in PostgreSQL.
- Fetch supported reports from Tally and cache only what is required for repeat queries.
- Record synchronization watermark, company identity, period, source and fetch time.
- Define freshness policy for each report and show stale-data warnings.
- Use private object storage for exports with short-lived signed links.
- Cache keys include tenant, connector, company, reporting period and relevant filters—not just company name.
- Consider a normalized reporting store/analytical database only if pilot measurements show a need.

6.4 Financial correctness

- Create sanitized test fixtures from representative Tally reports.
- Define date boundaries, debit/credit signs, opening/closing balances, rounding and grouping conventions.
- Compare every initial report against Tally output for agreed periods.
- Test empty, malformed, large, inconsistent and partial datasets.
- Do not silently substitute stale cached values; label freshness and fail safely when current data is required.

7. Initial Data Model and API Boundaries

7.1 Suggested entities

Entity	Purpose / key considerations
organizations	Tenant boundary; organization ID is primary tenant scope.
users	Identity record or reference to auth provider.
memberships	User-organization relationship, role, status and invitation lifecycle.
accounting_companies	Tally company identity linked to organization/connector; never rely solely on display name.
connectors	Installation ID, pairing status, version, health, heartbeat, revocation and credential metadata.
sync_runs	Start/end, status, company, report type, watermark, retries, errors and freshness.
report_runs	Report, parameters, company, requester, status, source metadata, result reference and timestamps.
chat_sessions / messages	Conversation history with organization/company scope and retention policy.
audit_events	Actor, organization, action, target, timestamp, outcome and redacted metadata.
approval_requests	Future-ready proposed action, payload hash, creator, approver, state and execution result; writes disabled in pilot.
generated_files	Private object key, tenant/company scope, owner, expiry/retention and checksum.

7.2 API boundaries (illustrative)

Endpoint family	Purpose
POST /organizations; GET /me	Onboarding and authenticated user context.
GET /companies; POST /connectors/pair	List authorized companies and pair connector securely.
GET /connectors/status	Health and last-seen information.
GET /reports/trial-balance	Validated trial balance query.
GET /reports/profit-loss	Validated P&L query.
GET /reports/balance-sheet	Validated balance sheet query.
GET /reports/outstanding-bills	Validated receivables/payables query.
GET /reports/ledger-statement	Authorized ledger statement query.
POST /chat/messages	Create finance question; server selects approved tools.
GET /report-runs/:id; GET /files/:id	Retrieve job/report status and authorized file downloads.

These are starting contracts, not a finalized API specification. Use a versioned API prefix, runtime validation, consistent errors, request IDs, pagination and idempotency where appropriate.

8. Security, Reliability and Operations

8.1 Mandatory controls

- Enforce organization/company permissions on every API request; use PostgreSQL RLS as defense in depth.
- Keep Tally XML port 9000 inaccessible from public internet.
- Use unique connector credentials, secure pairing, revocation, rotation and least privilege.
- Keep API keys/database secrets in server-side secret management; never commit secrets.
- Disable write-capable Tally tools in the pilot.
- Enable database backups and test restore procedures.

- Use TLS externally and appropriate encryption for sensitive stored data.
- Redact secrets and raw financial data from logs; restrict log access.
- Test tenant isolation, session expiry, authorization and report-download access.
- Document data retention, deletion, incident response and customer data access policies.

8.2 Reliability controls

- Connector heartbeat and last-successful-sync indicators.
- Bounded retries with exponential backoff and timeouts.
- Idempotency for sync/report jobs and duplicate-prevention rules.
- Durable job state and visible failed-job status.
- Structured logs with request/correlation IDs and redaction.
- Monitoring for API errors, connector health, queue lag, AI cost and report failures.
- Runbooks for connector offline, Tally unavailable, stale data, failed report and database restore.
- CI checks: lint, type checks, unit/integration tests, dependency checks and migration validation.

8.3 Production readiness definition

For this pilot, production-ready means controlled real-user access, reproducible financial outputs, secure tenant boundaries, recoverable operations, and no known critical defects in the agreed scope. It does not mean enterprise certification or a guarantee of zero defects.

9. Budget and Cost Model

9.1 One-time build budget

Workstream	Estimate
Codebase audit, requirements, architecture	■4,000–■7,000
Frontend and dashboard	■8,000–■15,000
Backend, database, authentication, tenancy	■12,000–■20,000
Tally connector adaptation and testing	■10,000–■18,000
Financial reports and AI orchestration	■8,000–■15,000
QA, security, deployment and handover	■6,000–■10,000
Estimated total	■48,000–■85,000

Constrained planning estimates for existing staff/founder-led work or limited specialist contracting, not vendor quotes. Reserve ■10,000–■15,000 for unexpected integration issues if the cash ceiling permits. If total spend must stay below ■1 lakh, keep scope tight.

9.2 Monthly operating costs

Planning scenario: 2–5 pilot businesses, modest report volume, no mobile app, WhatsApp or Tally write-back. Actual costs depend on provider, exchange rate, usage and region.

Cost item	Monthly planning range
Web hosting and API	■0–■2,500
PostgreSQL, authentication and backups	■0–■2,500
Domain, email and delivery	■200–■800
AI API usage	■500–■3,000
Error monitoring/logs	■0–■800
Private storage/exports	■0–■500
Miscellaneous/contingency	■500–■1,500
Indicative total	■1,200–■11,600/month

OPERATING TARGET
A reasonable initial target is ■3,000–■6,000/month after real users begin using the product. Free tiers may reduce early costs, but check uptime, retention, backups and limits before using them for real financial data.

9.3 Reference vendor prices to verify

- Supabase published pricing: Pro starts at US$25/month; Free plan has limits and may pause inactive projects. https://supabase.com/pricing
- Vercel published pricing: Pro starts at US$20/month; Hobby is intended for personal projects. https://vercel.com/pricing

Reference prices, not INR quotations. Verify current pricing, tax, currency conversion, region availability and included limits.

9.4 Illustrative AI usage calculation

Assumption: 3,000 input tokens and 500 output tokens per question. At the previously referenced example prices of US$2 per million input tokens and US$10 per million output tokens:

CALCULATION
(3,000 × $2 + 500 × $10) / 1,000,000 = US$0.011 per question. At 10,000 questions, this illustrative usage is US$110 before extra calls, retries or other charges.

This is a scenario, not a forecast. Model prices/availability can change; verify the chosen model's current price. Measure token usage and set per-user limits and budget alerts.

9.5 Engineering capacity is a separate cost

Team situation	Implication
Two or three developers already available	Best chance of meeting deadline within cash budget.
One strong full-stack developer plus part-time support	Possible if connector/report scope remains limited.
Freelancers hired for all work	Budget may require scope reductions or more time.
Agency end-to-end	Budget and timeline are aggressive for reliable live Tally product.

Do not treat cash budget as proof that all features can be delivered. Available engineering hours and integration skill are critical variables.

10. Thirty-Day Delivery Roadmap

Assumption: team can work in parallel on frontend, backend and connector. If one developer must do everything sequentially, reduce pilot scope.

Days 1–3 — Audit and discovery

- Clone and inspect E-Munshi and Tally MCP repositories.
- Identify stack, backend, database, deployment, license and dependency status.
- Run the existing app and document what works.
- Choose initial reports and pilot workflow.
- Define tenant, user, company, connector, report and audit data models.
- Write initial ADRs and threat model.

Exit criterion: Architecture approved, working development environments, and no unresolved license or connector feasibility blocker.

Days 4–7 — Foundation and Tally proof of concept

- Set up repositories, CI checks, environment configuration and deployments.
- Implement authentication, organization membership and database migrations.
- Build first Tally connection from a Windows development machine.

- Retrieve one report, such as trial balance, and compare it with Tally.
- Define the API contract between frontend and backend.

Exit criterion: Signed-in user can select a company and retrieve a verified report from a test Tally instance.

Days 8–14 — Financial engine and core UI

- Build connector pairing, health checks and retry handling.
- Implement initial report adapters.
- Build dashboard, report filters, loading/error/empty states.
- Add ledger and outstanding-bill views.
- Implement deterministic calculations and source metadata.
- Add organization-level authorization tests.

Exit criterion: Initial reports work end-to-end and totals/date ranges match agreed Tally reference outputs.

Days 15–21 — AI chat, exports and jobs

- Implement allowlisted finance tools and AI orchestration.
- Add clarification for ambiguous dates, companies and metrics.
- Build answer provenance, report exports and basic audit viewer.
- Add job retries, timeouts, idempotency and connector recovery.
- Implement first polished workpack only if core reports are stable.
- Test large, empty, malformed and inconsistent datasets.

Exit criterion: Supported questions return validated answers; unsupported requests fail safely; failures are visible and recoverable.

Days 22–26 — Security and pilot acceptance

- Test cross-tenant access, role enforcement, pairing and session expiry.
- Verify reports against agreed reference outputs from Tally.
- Test database backup and restore.
- Add production logging, alerts and rate limits.
- Package connector and test on a clean Windows machine.
- Onboard one internal or friendly pilot customer.

Exit criterion: No known critical security defects, report reconciliation passes, and installation is repeatable.

Days 27–30 — Controlled pilot launch

- Onboard remaining pilot users.
- Monitor answer quality, connector uptime, API failures and costs.
- Fix highest-severity bugs.
- Write user onboarding and support documentation.
- Record known limitations and next development backlog.

Exit criterion: Small group of real customers can connect Tally, generate reports, ask supported questions, and recover from common errors.

10.1 Team ownership

Role / responsibility	Primary ownership
Backend/integration developer	Tally connector, APIs, data model, financial engine and AI tools.
Frontend developer	Dashboard, chat, report pages, onboarding and UX.
QA/DevOps or shared engineer	Test datasets, CI/CD, tenant isolation, monitoring, backup/restore and release.
Product owner	Scope decisions, customer interviews, report definitions and acceptance criteria.

These are responsibilities, not necessarily four full-time hires. People can own multiple areas, but security and financial-correctness testing must have explicit ownership.

11. Feature Acceptance Criteria

Feature	Definition of done
Connect Tally	Customer pairs connector and sees correct company and connection health.
Financial reports	Totals match Tally output for same company and agreed periods.
AI finance chat	Questions map to permitted tools; ambiguity triggers clarification; unsupported requests fail safely.
Source tracing	Figures trace to report results or relevant ledger/voucher references.
Tenant security	A user cannot read another organization's records by changing URL/API identifiers.
Report export	Company, date range, figures and totals match on-screen report.
Recovery	Interrupted sync retries without duplicating/corrupting data.
Production deployment	HTTPS, managed secrets, backups, error reporting and tested recovery are in place.

12. Using AI to Build the Product Safely

- Work from written requirements and accepted ADRs.
- Implement small changes with explicit acceptance criteria.
- Generate unit tests alongside financial calculations.
- Manually review security-sensitive changes.
- Keep secrets, production data and credentials out of prompts.
- Run linting, type checks, automated tests and integration tests before merging.
- Do not allow an AI agent to independently change database schema, connector security and authentication without review.
- Optimize for code the team can understand, test, debug and maintain—not code volume or speed of initial generation.

13. Immediate Next Steps and Open Questions

13.1 First three engineering tasks

Task 1 — Audit both repositories

Deliverable: codebase audit and keep/adapt/replace/build-from-scratch list. Establish stack, run the app, inspect MCP license/source, and identify production gaps.

Task 2 — Prove the Tally integration

Deliverable: working, repeatable, read-only integration on a test Windows machine, with reference dataset and report totals verified against Tally. Test reconnection after failure.

Task 3 — Build one end-to-end journey

Deliverable: sign in → select company → view trial balance → ask supported question → receive verified answer with source information, crossing frontend, backend, database, AI and connector.

13.2 Risks and unresolved questions

Question / risk	Why it matters	Resolution
MCP license and code quality	Determines reuse and rewrite scope.	Inspect license, source, dependencies, auth, caching and write tools on days 1–3.
Team capacity and skills	Determines timeline realism.	Confirm team count, TypeScript/React skills, Windows/Tally experience and hours.
Tally customer environments	Configuration affects data and connector behaviour.	Test representative TallyPrime installations and company data.
Financial definitions	Date/sign/opening/closing balance semantics must be agreed.	Define report specs and compare with Tally reference outputs.
Pilot customer count	Changes support, tenant testing and onboarding load.	Choose small cohort and agree success criteria.
Data retention and privacy	Financial records are sensitive.	Set retention, deletion, support-access and incident policies before live onboarding.

13.3 Information needed from the team

- E-Munshi GitHub repository, or framework/backend/database/deployment details.
- Number of developers, primary skills and hours available over 30 days.
- Whether pilot customers use TallyPrime locally on Windows and whether test access is available.
- Target pilot count and specific reports/questions needed first.
- Whether ■50,000–■1,00,000 includes developer compensation or only incremental project expenses.

14. Final Recommendation

RECOMMENDED PLAN
Build E-Munshi as a TypeScript modular monolith with PostgreSQL, a read-only Windows Tally connector, deterministic financial calculations, and controlled AI tool orchestration. Reuse MCP integration logic only after license and code review. Keep the pilot narrow, measurable and auditable.

The first-month goal is a working pilot with real Tally data and a small set of reliable financial workflows—not a promise to reproduce every Riko feature in 30 days.

Resolve three uncertainties before committing to full scope: (1) repository license/source and intended reuse; (2) engineering capacity; and (3) accuracy/availability of required reports on pilot customers' TallyPrime configurations.

Once those are proven, the rest of the plan becomes more predictable.

Reference Links

- Riko AI: https://rikoai.in/
- Riko AI features: https://rikoai.in/features
- Riko AI FAQ: https://rikoai.in/faq
- E-Munshi prototype: http://emunshie.com/
- Tally MCP server repository: https://github.com/dhananjay1405/tally-mcp-server.git
- Supabase pricing: https://supabase.com/pricing
- Vercel pricing: https://vercel.com/pricing

Pricing, product pages, and repository contents can change. Re-verify vendor prices, feature details, licensing, and source code before implementation or purchase.